

Sistema de Gestión de Inventario TechLab

1. Introducción

El documento detalla la solución técnica de **TechLab**, un sistema diseñado con el propósito de facilitar la gestión del inventario (operaciones CRUD) y permitir la administración de pedidos .

Para lograr una implementación que sustente este propósito el desarrollo se apoyó en el paradigma de **Orientación a Objetos** y en la aplicación de algunos principios de diseño. Se prioriza el **encapsulamiento** mediante la **Ley de Demeter** (mínimo conocimiento) y el patrón **'Tell, don't ask'**, mientras que el uso de **DRY** (*Don't Repeat Yourself*) y el **Principio de Mínimo Asombro** aseguran que el código sea predecible y escalable. Esta estructura garantiza una separación entre la lógica de negocio y la interfaz de usuario, facilitando el mantenimiento evolutivo del sistema.

2. Supuestos

Para el desarrollo del programa se consideraron los siguientes supuestos funcionales y técnicos:

- **Identificadores únicos:** Cada producto posee un ID numérico único generado automáticamente para evitar colisiones de datos y facilitar la búsqueda o identificación, incluso tras la carga de datos externos.
- **Integridad de Datos:** No se permiten productos con valores negativos en precio o stock; cualquier intento de ingreso de datos inválidos es bloqueado por el sistema mediante excepciones.
- **Consistencia de Pedidos:** Un pedido sólo puede confirmarse si existe stock suficiente de todos los productos solicitados al momento de la venta.
- **Persistencia Local:** El archivo `productos.json` se considera la fuente de verdad inicial para el inventario al momento de iniciar la aplicación.

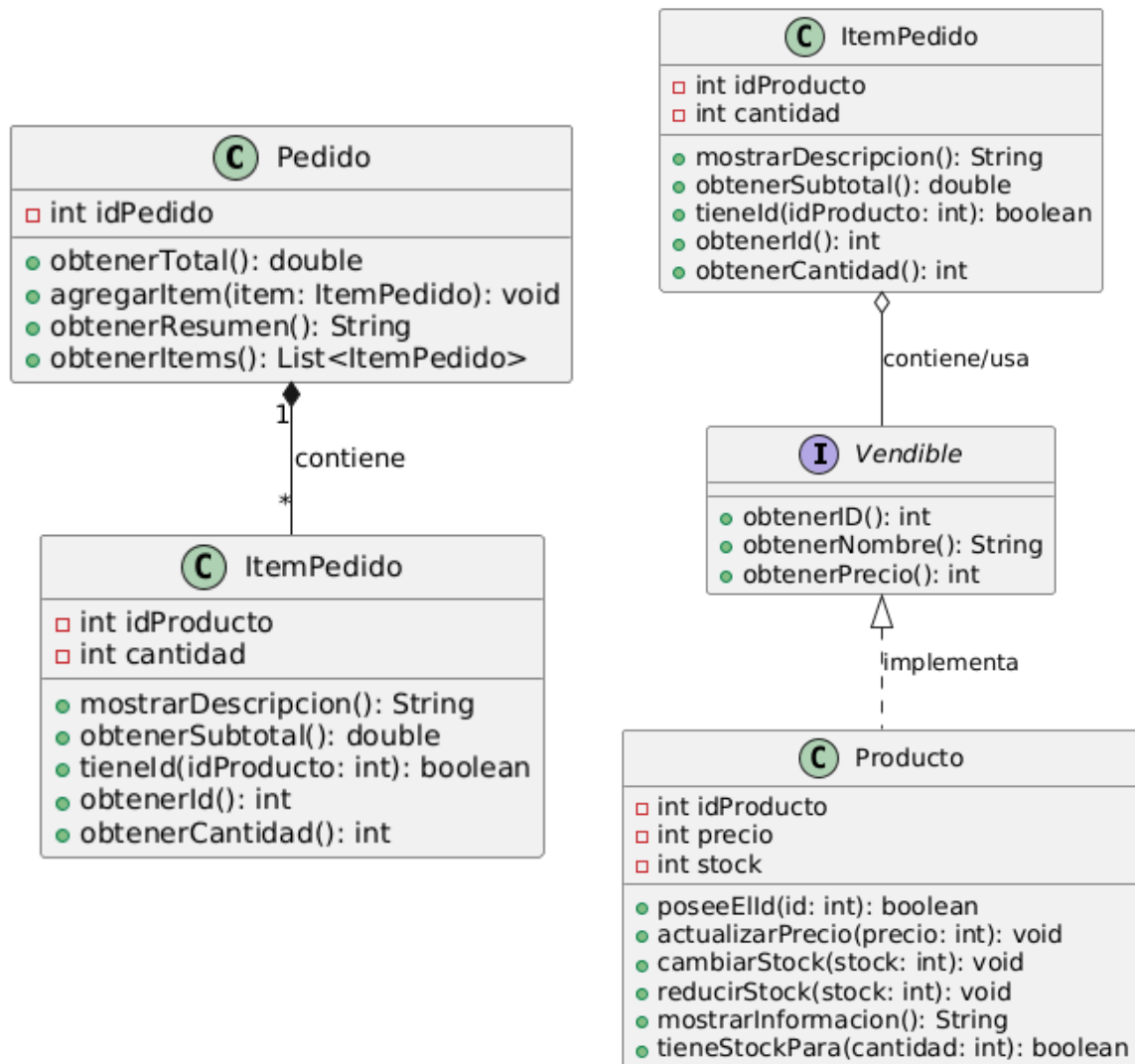
3. Modelo de Dominio

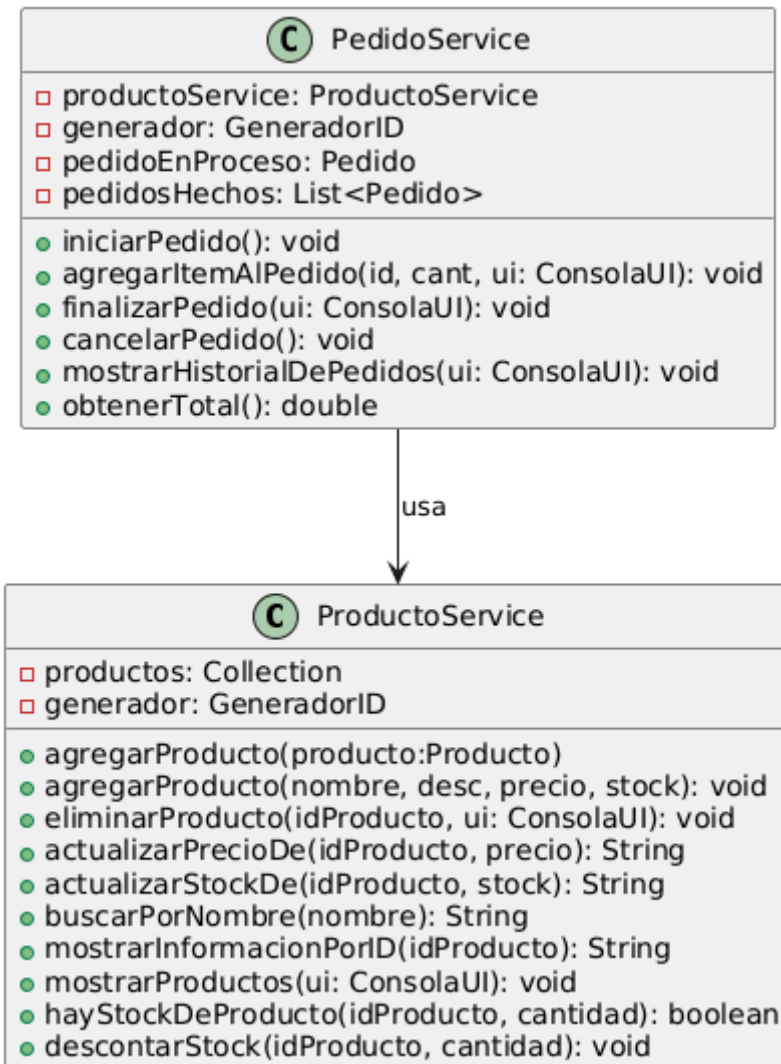
El sistema está organizado en capas para maximizar el bajo acoplamiento:

- **Capa de Servicios:** Clases como `ProductoService` y `PedidoService` actúan como controladores de la lógica de negocio. Estas clases no conocen la consola; reciben una abstracción que funciona como interfaz o impresora para el usuario.
- **Capa de Modelos:** Entidades como `Producto`, `Pedido` e `ItemPedido` representan los datos del sistema. Se utiliza la interfaz `Vendible` para abstraer a un

ItemPedido de la interfaz pública que ofrece **Producto** logrando con esto que **ItemPedido** no tenga acceso a nada que no le importe como los métodos para cambiar precio o stock de producto. Si a futuro cambia la implementación de **Producto** entonces **ItemPedido** se mantiene al resguardo al depender únicamente de abstracciones.

4. Diagramas de Clase





5. Detalles de Implementación

5.1. Inyección de Dependencias y Desacoplamiento UI

Una decisión de diseño fue la **inyección de la interfaz de usuario** en los servicios. En lugar de que **ProductoService** utilice `System.out.println`, este recibe un objeto **ConsolaUI** por parámetro. Esto permite que la lógica de gestión de productos o pedidos sea totalmente independiente de la forma en que se muestran los datos.

5.2. Sincronización de Identificadores

Para manejar la transición entre productos cargados por JSON y productos nuevos, el **GeneradorID** implementa un método de **sincronización**. Al cargar el inventario inicial, el generador analiza los IDs existentes y ajusta su contador interno para asegurar que el próximo producto creado manualmente tenga un ID correlativo y único.

5.3. ProductoService: Manejo de fuente de verdad

- **Fuente de Verdad:** Actúa como el repositorio central y única fuente de verdad para los productos en memoria, gestionando la colección principal y la integridad de los identificadores únicos mediante el **GeneradorID**.
- **Gestión CRUD:** Centraliza todas las operaciones de administración, permitiendo el alta, baja y modificación de productos, además de implementar lógica de búsqueda por nombre e ID.
- **Proveedor de Datos:** Sirve información procesada a otros componentes mediante la interfaz **Vendible**, permitiendo que el sistema sea escalable y desacoplado de la representación visual.

5.4. PedidoService: Gestor del Ciclo de Venta

- **Control de Estado:** Administra el flujo completo de un pedido, manteniendo el estado desde su apertura hasta su cierre o anulación.
- **Orquestación de Ítems:** Se encarga de la construcción dinámica del pedido, permitiendo agregar ítems mientras valida la disponibilidad de stock en tiempo real mediante consultas al **ProductoService**.
- **Impacto en Inventario:** Una vez confirmado el pedido, el servicio calcula el resumen final (delegando a las entidades que manejan el estado correspondiente) y coordina con el **ProductoService** para realizar la **reducción efectiva de stock**, cerrando así el ciclo de negocio.

6. Excepciones

El sistema implementa un manejo de errores mediante excepciones personalizadas y de sistema:

- **StockInvalidoException / PrecioInvalidoException:** Lanzadas por el modelo **Producto** para evitar estados inconsistentes.
- **IdNoEncontradoException:** Utilizada por los servicios cuando se intenta operar sobre un producto inexistente.